

A Comparative Performance Analysis between AWS Lambda-Based Serverless Architecture and Traditional On-Premise Architecture for Real-Time Data Pipelines

Tan Yi Ya

Faculty of Computing, Universiti Teknologi Malaysia

Johor Bahru, Malaysia

tanyiya@graduate.utm.my

Abstract— This literature review investigates the comparative performance between AWS Lambda-based serverless architectures against traditional on-premise architectures for real-time data pipelines. The growth in the amount of digital data increases real-time data processing needs, while traditional batch processing is inadequate for real-time responsiveness and intelligent decision-making. The paper analyses the architecture, characteristics, and performance of AWS Lambda and compares both approaches using metrics such as latency, throughput, scalability, cost, and security. The findings indicate that AWS Lambda serverless computing architecture provides superior scalability, throughput and cost efficiency, making it suitable for modern and event-driven applications, while traditional systems offer greater control but face higher operational overhead and limited flexibility.

Keywords— AWS Lambda, serverless architecture, real-time data pipelines, ETL, performance benchmarking.

I. INTRODUCTION

In the current era of information age, the exponential growth of digital data has caused a trend in IT infrastructure that focuses on efficient, automated and decentralized data processing models. Many organisations are moving away from the monolithic legacy systems and adopting cloud-native architecture to remain competitive in the digital economy. Traditional data processing architectures often rely on batch processed ETL (Extract, Transform, Load) pipelines which are too slow for real time needs such as fraud detection or real time analytics. Moreover, on-premises architectures also suffer from problems such as low resource utilization and high maintenance costs. Replacing traditional architecture with serverless computing can reduce capital expenditure and allow enterprises to focus on business decisions rather than server management. This paper analyses the characteristics, performance and limitations of AWS Lambda serverless computing architecture, then compares it with the traditional on premise architecture on building data pipelines.

II. BACKGROUND

AWS Lambda is a serverless computing provider that allows users to use event-related services without managing servers. (Thokala & Gupta, 2025) The term ‘serverless’ refers to the Function-as-a-Service provider, in which the infrastructure layer is fully abstracted away (Pogiatzis & Samakovitis, 2021), scaling automatically based on the amount of resources used and utilizing a pay-as-you-go pricing model billed in 1 ms increment. (Mathew et al., 2021)

III. ARCHITECTURE

A serverless real-time data pipeline typically consists of the following components:

- 1) *Data Source*: AWS S3 bucket acts both as a data source and datalake, providing low latency and out-of-the-box lambda integration capabilities
- 2) *Triggering*: Event notification SNS topics and SQS queues are used to invoke the processing stage of lambda function.
- 3) *Data Processing*: Stateless AWS Lambda functions are used to process incoming data, simplifying scalability but limiting operations that require shared or global state.
- 4) *Data Writing*: Processed data is sent through SQS queues to Lambda functions that handle persistence in DynamoDB, with outputs split into smaller chunks (≤ 256 KB) to support parallel writing and improve fault tolerance.
- 5) *Data Sink*: Amazon DynamoDB serves as the final storage layer, offering a serverless, low-latency NoSQL solution with on-demand pricing, built-in deduplication, fast querying, and easy integration with additional pipelines via DynamoDB Streams. (Pogiatzis & Samakovitis, 2021)

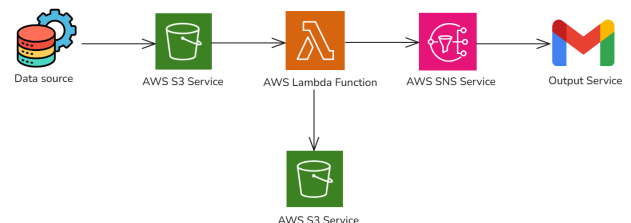


Fig. 1 Example of AWS Serverless Data Pipeline Architecture (Jadhav et al., 2024)

IV. CHARACTERISTICS OF AWS LAMBDA

An important feature of AWS Lambda serverless computing is the ability to downsize to nothing, ensuring that customers will not face penalties for idleness when resources are not utilized effectively. (Verma et al., 2024) Lambda functions use external databases to share state and coordinate workflows. (Mathew et al., 2021) Furthermore, serverless components are very elastic allowing for almost instant horizontal and vertical scaling. (Jadhav et al., 2024)

V. REAL WORLD EXAMPLE

Netflix uses AWS Lambda for their serverless, event-driven real-time data pipeline, for automation workflows and media processing. AWS Lambda functions are triggered by events such as file uploads to Amazon S3 to automatically process, encode and distribute video content without managing servers. For example, Netflix used AWS Lambda functions to automate the tagging of newly created resources. This approach improved cost visibility, accountability, promoted efficient resource allocation and streamlined auditing and compliance of Netflix. (Singh, 2024)

VI. COMPARATIVE ANALYSIS

This section compares AWS Lambda-based serverless pipelines architecture and traditional on-premise architectures by the following metrics:

- 1) *Latency*: This metric represents the time taken by a system to complete its operation (Chikhalkar et al., 2025) defined by backward length of time from when a function is triggered until it completes its task. (M et al., 2025) An ideal architecture is one with low latency.
- 2) *Throughput*: This metric measures the system's capability of processing requests per second. An ideal architecture is one with high throughput.
- 3) *Scalability*: This metric measures the ability to handle increasing workload without lowering performance.
- 4) *Cost*: This metric measures chargeable elements scale across pipeline runs (Pogiatzis & Samakovitis, 2021). An ideal architecture should cost lower in the long term.
- 5) *Security*: This metric evaluates an architecture's ability to protect data, workloads and system resources from vulnerabilities. (Boggavarapu, n.d.)

TABLE I
COMPARISON TABLE BETWEEN ON-PREMISE ARCHITECTURE AND AWS LAMBDA SERVERLESS ARCHITECTURE FOR DATA PIPELINES

Metric	Traditional On-Premise	AWS Lambda Serverless
Latency	Consistent response times (~120ms) (M et al., 2025)	Exhibits "Cold Start" delays (300–600ms) but later achieve steady-state latency (~60ms). (M et al., 2025)
Throughput	Suffers performance degradation to 800 requests per second at high loads (>1200 req/sec). (M et al., 2025)	Achieves near-linear scaling up to 5000+ requests per second with ~98% throughput efficiency. (M et al., 2025)
Scalability	Challenges in scalability mentioned by multiple sources. (Boggavarapu, n.d.) (Chikhalkar et al., 2025)	Highly elastic, allows near instant horizontal and vertical scaling. (Jadhav et al., 2024)
Cost	Fixed server pricing. Expensive due to idle resource waste and high maintenance costs. (M et al., 2025)	Pay-per-use pricing model. According to (M et al., 2025), serverless architecture is 50% more cost-effective for fluctuating or bursty workloads.
Security	Full control over infrastructure, enabling customized security policies and isolation. (M et al., 2025)	Shared responsibility model. Cloud provider secures infrastructure while users secure code and configurations. (Pogiatzis & Samakovitis, 2021)

VII. CONCLUSION

In conclusion, AWS Lambda serverless architecture is an efficient alternative to traditional on-premise architecture on deploying data pipeline systems. While traditional architectures provide full control over infrastructure, they suffer from limitations such as high operational costs, limited scalability and slower adaptability to dynamic workloads, and are highly dependent on their own organisation to manage the resource effectively. In contrast, AWS Lambda has superior performance in throughput and scalability, along with a more cost-effective pay-per-use model, making it

well-suited for real-time and event-driven data pipeline applications.

However, the shift to AWS Lambda Architecture includes challenges such as cold start latency and reduced control over the infrastructure. Despite the limitations, serverless computing represents a significant advancement in cloud-native designed data pipeline. Overall, AWS Lambda is a compelling choice for modern data pipelines, provided that the limitations are properly managed and aligned with organizational requirements.

VIII. FUTURE RESEARCH DIRECTIONS

Future research should explore hybrid computing models that dynamically distribute workloads between cloud and edge environments to optimize latency, cost, and data locality in real-time data pipelines. Additionally, multi-cloud serverless strategies should be compared by their benchmarking platforms such as AWS Lambda, Azure Functions and Google Cloud Functions to provide a more comprehensive comparison across vendors.

IX. REFERENCES

- Boggavarapu, V. (n.d.). Modernizing Legacy Systems with Cloud-Native Data Architectures: Case Studies in Banking. *Journal of Computer Science and Technology Studies*, 176-186. 10.32996/jcsts
- Chikhalkar, A., Brüninghaus, M., Deppe, S., Bicker, E., & Röcker, C. (2025). A Data Pipeline Concept for Digitizing Services in Small and Medium-Sized Companies. *INTERNATIONAL JOURNAL ON INFORMATICS VISUALIZATION*, 9(1), 333-341.
- Jadhav, R., Lunawat, A., Mali, K., More, P., & Varade, D. P. (2024). ETL PIPELINE USING LAMBDA SERVICES. *2024 Intelligent Systems and Machine Learning Conference (ISML)*, 32-37. 10.1109/ISML60050.2024.11007433
- M, V., B, S., Seidakhmet, N., Syed Riyaz, A., M C, P., & E, N. (2025). Cloud Computing and Serverless Architectures Innovations and Applications. *13th IEEE International Conference on Smart Grid*, 160-167. 10.1109/ICSMARTGRID66138.2025.11071764
- Mathew, A., Andrikopoulos, V., & Blaauw, F. J. (2021). Exploring the cost and performance benefits of AWS Step Functions using a data processing pipeline. *IEEE/ACM 14th International Conference on Utility and Cloud Computing*. 10.1145/3468737.3494084
- Pogiatzis, A., & Samakovitis, G. (2021). An Event-Driven Serverless ETL Pipeline on AWS. *mdpi*. 10.3390/app11010191
- Singh, A. (2024, October 31). *AWS Case Study: Netflix's Automated Tagging Strategy for Cost Optimization*. Arun's Substack. <https://dyota.substack.com/p/aws-case-study-netflixs-automated>
- Thokala, V. S., & Gupta, S. (2025). Analysing the Impact of Cloud-Native Architectures on Web Application Performance Using AWS Lambda. *2025 International Conference on Advancements in Smart, Secure and Intelligent Computing (ASSIC)*. 10.1109/ASSIC64892.2025.11158212
- Verma, P., Goel, P., & Rani, N. (2024). A Review: Cold Start Latency in Serverless Computing. *2024 Sixth International Conference on Computational Intelligence and Communication Technologies (CCICT)*, 141-148. 10.1109/CCICT62777.2024.00034